

Master Engineering Blueprint: School Website & PVC ID Card Studio

This master operational manual provides a hyper-granular, week-by-week task breakdown for **Developer A (You)** and **Developer B (Friend)**. It details exact file paths, logical implementation steps, exact terminal commands, and technical solutions for every single phase of development.

To maximize your friend's learning experience, **Developer B** handles advanced CSS architectures, JavaScript DOM event binding, browser file memory, print-media engines, and backend integrations. **Developer A** oversees repository governance, semantic HTML foundations, validation rules, and responsive scaffolding.

Quick-Reference Terminal Command Cheat Sheet

Keep this command reference handy for your daily and weekly Git workflows:

- **Download repository:** `git clone [https://github.com/YOUR-USERNAME/school-website.git](https://github.com/YOUR-USERNAME/school-website.git)`
- **Check modified files:** `git status`
- **Stage all changes:** `git add .`
- **Commit changes with message:** `git commit -m "descriptive message here"`
- **Upload changes to GitHub:** `git push origin main`

- **Download latest partner updates:** `git pull origin main`

MONTH 1: Repository Architecture, DOM Binding & CR80 Print Engine

- **Monthly Aim:** Establish professional version control workflows, build structural HTML layouts, and code a real-time form-to-preview ID card generator optimized for physical PVC printer output.

Week 1: Git Foundations, Directory Tree & Wireframing

- **Aim of the Week:** Configure the remote repository, establish local development environments, and map out UI wireframes.
- **Potential Roadblock:** Authentication errors or permission denied when pushing to GitHub.
- **Resolution / Command:** Use GitHub CLI auth or personal access tokens: `gh auth login`

Developer A (You)

- **What to do:** Initialize the repository, configure tracking security, and push the base folder scaffolding.
- **Where to do it:** Local terminal & GitHub web interface.
- **How to do it (Step-by-Step):**
 1. Create a public repository named `school-website` on GitHub with a `README.md` and a Node `.gitignore`.

2. Open your local terminal and clone the repository:

Bash



```
git clone https://github.com/YOUR-USERNAME/school-website.git
cd school-website
```

3. Create the foundational directory structure using terminal commands:

Bash



```
mkdir css js assets assets/images
touch index.html academics.html faculty.html id-card.html
```

4. Stage, commit, and push the directory tree:

Bash



```
git add .
git commit -m "infra: setup initial directory structure and
skeleton html files"
git push origin main
```

5. Go to GitHub repository **Settings** > **Collaborators** > **Add people**, enter your friend's username, and send the invitation link.

Developer B (Friend)

- **What to do:** Accept repository collaboration, clone the project locally, design UI wireframes, and calculate CR80 print dimensions.

- **Where to do it:** Figma (free tier at figma.com) & local workspace.

- **How to do it (Step-by-Step):**

1. Accept the email invite to become a repository collaborator.

2. Clone the repository onto your local computer:

Bash



```
git clone https://github.com/YOUR-USERNAME/school-website.git
cd school-website
```

3. Open [Figma](#) and design two primary layouts:

- *Layout 1:* School landing page (`index.html`) featuring a hero banner, navigation bar, and notice board.
- *Layout 2:* ID Card Maker studio (`id-card.html`) with a split screen (form inputs on the left, live preview card on the right).

4. Document physical ISO/IEC 7810 ID-1 / CR80 card dimensions ($85.60 \text{ mm} \times 53.98 \text{ mm}$), or roughly $1013\text{px} \times 638\text{px}$ at 300 DPI) for print scaling calculations.

5. Test pushing a minor change to verify write access:

Bash



```
git status
git add README.md
git commit -m "docs: added wireframe dimension notes"
git push origin main
```

- **Weekend Sync (Friday/Saturday):** Join a quick call. Both partners run `git pull origin main` to ensure local environments match. Review wireframes together.

Week 2: Input Form Architecture & Global Stylesheet Structuring

- **Aim of the Week:** Write semantic HTML markup for the school pages and build the CSS design system and ID card input form.
- **Potential Roadblock:** CSS style conflicts from unorganized global styles.
- **Resolution / Command:** Use CSS custom variables and isolated stylesheet declarations.

Developer A (You)

- **What to do:** Code the structural layout of `academics.html` and write the mobile navigation toggle script.
- **Where to do it:** `academics.html` and `js/main.js`.
- **How to do it (Step-by-Step):**

1. Open `academics.html` in VS Code and write semantic HTML5 markup:

HTML



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>School Academics Portal</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <nav class="navbar">...</nav>
  <main class="content">
    <h1>Academic Programs & Curriculum</h1>
  </main>
  <script src="js/main.js"></script>
</body>
</html>
```

2. Create `js/main.js` and write a simple mobile hamburger menu toggle event listener.

3. Save, commit, and push:

Bash



```
git add .
git commit -m "feat: added academics page markup and mobile
nav script"
git push origin main
```

Developer B (Friend)

- **What to do:** Build the global design system stylesheet (`css/style.css`) and construct the complete ID card input form (`id-card.html`).
- **Where to do it:** `css/style.css` and `id-card.html`.
- **How to do it (Step-by-Step):**

1. Open `css/style.css` and define CSS custom variables:

CSS



```
:root {
  --primary-blue: #1e3a8a;
  --accent-gold: #d97706;
  --bg-light: #f3f4f6;
  --text-dark: #1f2937;
}
body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; margin: 0; background: var(--bg-light); color: var(--text-dark); }
```

2. Open `id-card.html` and construct the form markup with native elements: text fields, class selectors, and an image file upload picker (`<input type="file" accept="image/*" id="avatar-input">`).

3. Stage and push your changes:

Bash



```
git add css/style.css id-card.html
git commit -m "feat: built global design variables and id card
input form"
git push origin main
```

- **Weekend Sync:** Pull updates (`git pull origin main`), verify that global styles apply correctly to all HTML files, and check navigation links between pages.

Week 3: Real-Time DOM Data Binding & CSS Print Media Engine

- **Aim of the Week:** Connect form inputs to instantly update a live visual preview card and configure printer-ready CSS rules for physical PVC CR80 card stock.
- **Potential Roadblock:** Background colors or graphics disappearing during browser print execution.
- **Resolution / Command:** Enforce print color accuracy using `-webkit-print-color-adjust: exact; print-color-adjust: exact;` inside CSS print media queries.

Developer A (You)

- **What to do:** Build the responsive media query stylesheet to handle layout adjustments across viewports.
- **Where to do it:** `css/responsive.css`.
- **How to do it (Step-by-Step):**

1. Create `css/responsive.css` and write media queries targeting mobile and tablet widths:

CSS



```
@media (max-width: 768px) {  
  .id-studio-container {  
    flex-direction: column;  
  }  
}
```

2. Link `responsive.css` inside the `<head>` of your HTML files.

3. Commit and push:

Bash



```
git add css/responsive.css  
git commit -m "feat: added responsive layout media queries"  
git push origin main
```

Developer B (Friend)

- **What to do:** Write real-time JavaScript DOM data-binding logic (`js/id-generator.js`) and author strict print stylesheet rules (`css/id-card.css`).
- **Where to do it:** `js/id-generator.js` and `css/id-card.css`.
- **How to do it (Step-by-Step):**

1. Create `js/id-generator.js` and write event listeners capturing form inputs to update the visual preview card in real time:

JavaScript



```
const nameInput = document.getElementById('name-input');
const previewName = document.getElementById('preview-name');
nameInput.addEventListener('input', (e) => {
    previewName.textContent = e.target.value || 'Student
Name';
});
const avatarInput = document.getElementById('avatar-input');
const previewAvatar = document.getElementById('preview-
avatar');
avatarInput.addEventListener('change', (e) => {
    const file = e.target.files[0];
    if (file) {
        previewAvatar.src = URL.createObjectURL(file);
    }
});
```

2. Create `css/id-card.css` and write strict `@media print` rules to isolate the ID card container during printing:

CSS



```
@media print {
  body * { visibility: hidden; }
  #id-card-preview, #id-card-preview * { visibility: visible; }
  #id-card-preview {
    position: absolute;
    left: 0;
    top: 0;
    width: 3.375in;
    height: 2.125in;
    -webkit-print-color-adjust: exact;
    print-color-adjust: exact;
  }
}
```

3. Stage, commit, and push:

Bash



```
git add js/id-generator.js css/id-card.css
git commit -m "feat: implemented real-time dom data binding
and print media stylesheet"
git push origin main
```

- **Weekend Sync:** Pull updates, test typing student data into the form to ensure the preview updates instantly, and execute a virtual PDF print test.

Week 4: Template Switchers, Validation & Version 1.0 Alpha Release

- **Aim of the Week:** Implement multiple visual ID card templates and theme color pickers, concluding with a stable Version 1.0 merge.
- **Potential Roadblock:** Missing required field data during print generation.

- **Resolution / Command:** Implement client-side input validation checking before triggering `window.print()`.

Developer A (You)

- **What to do:** Write client-side validation logic ensuring mandatory form fields are populated before triggering print actions.
- **Where to do it:** `js/id-generator.js`.
- **How to do it (Step-by-Step):**

1. Open `js/id-generator.js` and add a print validation check function:

JavaScript



```
const printBtn = document.getElementById('print-btn');
printBtn.addEventListener('click', (e) => {
  if (!nameInput.value) {
    alert('Please enter a student name before printing!');
    e.preventDefault();
    return;
  }
  window.print();
});
```

2. Commit and push:

Bash



```
git add js/id-generator.js
git commit -m "feat: added input validation check for print
actions"
git push origin main
```

Developer B (Friend)

- **What to do:** Code template switcher buttons and write the project `README.md` file.
- **Where to do it:** `id-card.html`, `js/id-generator.js`, and `README.md`.
- **How to do it (Step-by-Step):**
 1. Add template selection buttons to `id-card.html` (e.g., Modern Style vs. Classic Style).
 2. Write JavaScript logic in `js/id-generator.js` to toggle modifier classes (`.template-modern`, `.template-classic`) on the card preview container.
 3. Open `README.md` and write complete project documentation detailing installation, file structure, and local server startup steps.
 4. Stage, commit, and push:

Bash



```
git add .  
git commit -m "feat: added card template switchers and updated  
readme documentation"  
git push origin main
```

- **Weekend Sync:** Conduct a full end-to-end walkthrough test. Tag the repository release: `git tag v1.0-alpha` and `git push origin v1.0-alpha`.

MONTH 2: Multi-Page Expansion & Local Data Persistence

- **Monthly Aim:** Build out the faculty directory interface and implement browser storage mechanisms to save generated student records locally.

Week 5: Faculty Directory Portal Integration

- **Aim of the Week:** Construct a responsive teacher directory page using advanced CSS Grid configurations.
- **Potential Roadblock:** Inconsistent card grid layouts on medium screen viewports.
- **Resolution / Command:** Use CSS Grid auto-fill with `minmax()` boundaries (`grid-template-columns: repeat(auto-fill, minmax(250px, 1fr))`).

Developer A (You)

- **What to do:** Finalize static page links and header navigation consistency across all HTML files.
- **Where to do it:** `index.html`, `academics.html`, `faculty.html`, `id-card.html`.
- **How to do it (Step-by-Step):**
 1. Audit all navigation `<nav>` blocks across every HTML file to guarantee uniform relative paths.
 2. Commit and push:

Bash



```
git add .  
git commit -m "fix: standardized navigation headers across all  
html files"  
git push origin main
```

Developer B (Friend)

- **What to do:** Build `faculty.html` using advanced CSS Grid and card component styling with hover effects.
- **Where to do it:** `faculty.html` and `css/style.css`.
- **How to do it (Step-by-Step):**
 1. Open `faculty.html` and structure teacher profile cards inside a container.
 2. Open `css/style.css` and write grid rules:

CSS



```
.faculty-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px,
1fr));
  gap: 2rem;
  padding: 2rem;
}
.faculty-card {
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  transition: transform 0.3s ease;
}
.faculty-card:hover { transform: translateY(-5px); }
```

3. Commit and push:

Bash



```
git add faculty.html css/style.css
git commit -m "feat: built responsive faculty directory grid
and hover effects"
git push origin main
```

- **Weekend Sync:** Review grid responsiveness and check cross-page navigation links.

Week 6: Browser LocalStorage Record Archiving

- **Aim of the Week:** Implement browser storage mechanisms to save generated student records locally into JSON arrays.
- **Potential Roadblock:** Storage capacity limits or string serialization failures.

- **Resolution / Command:** Use `JSON.stringify()` when saving and `JSON.parse()` when reading from `localStorage`.

Developer A (You)

- **What to do:** Design the user interface layout elements for viewing saved student records in a tabular dashboard view.
- **Where to do it:** `id-card.html` (adding an archive records table section).
- **How to do it (Step-by-Step):**

1. Add an archive table container element to `id-card.html`:

HTML



```
<div id="archive-section">
  <h2>Saved Student Records</h2>
  <table id="archive-table">
    <thead><tr><th>Name</th><th>Roll No</th><th>Class</th>
    <th>Action</th></tr></thead>
    <tbody></tbody>
  </table>
</div>
```

2. Commit and push:

Bash



```
git add id-card.html
git commit -m "feat: added archive records table layout container"
git push origin main
```

Developer B (Friend)

- **What to do:** Write JavaScript storage utilities utilizing `localStorage` to serialize and render saved student ID records.

- **Where to do it:** `js/id-generator.js`.

- **How to do it (Step-by-Step):**

1. Open `js/id-generator.js` and write functions to save record state:

JavaScript



```

const saveBtn = document.getElementById('save-record-btn');
saveBtn.addEventListener('click', () => {
  const student = {
    name: nameInput.value,
    rollNo: document.getElementById('roll-input').value,
    classVal: document.getElementById('class-input').value
  };
  let records = JSON.parse(localStorage.getItem('studentRecords')) || [];
  records.push(student);
  localStorage.setItem('studentRecords',
    JSON.stringify(records));
  loadArchiveTable();
});
function loadArchiveTable() {
  const tbody = document.querySelector('#archive-table tbody');
  tbody.innerHTML = '';
  let records = JSON.parse(localStorage.getItem('studentRecords')) || [];
  records.forEach((rec, index) => {
    tbody.innerHTML += `<tr><td>${rec.name}</td>
<td>${rec.rollNo}</td><td>${rec.classVal}</td><td><button
onclick="deleteRecord(${index})">Delete</button></td></tr>`;
  });
}
window.onload = loadArchiveTable;

```

2. Commit and push:

Bash



```

git add js/id-generator.js
git commit -m "feat: implemented localStorage archiving and
table rendering logic"
git push origin main

```

- **Weekend Sync:** Test record saving workflows, reload browsers, and verify persistent data integrity.

Week 7: Mobile Viewport Optimization & Scaled Canvas Previews

- **Aim of the Week:** Ensure the ID Card Maker and website scale seamlessly across mobile phones and tablet viewports.
- **Potential Roadblock:** ID card preview overflowing small mobile device viewports.
- **Resolution / Command:** Apply CSS `transform: scale()` matrices to shrink high-resolution card layouts proportionally on smaller screens.

Developer A (You)

- **What to do:** Audit mobile form inputs to ensure touch targets are appropriately padded and virtual keyboard overlays do not disrupt layout flow.
- **Where to do it:** `css/responsive.css`.
- **How to do it (Step-by-Step):**

1. Update `css/responsive.css` to add touch padding rules for mobile inputs:

CSS



```
@media (max-width: 768px) {  
  input, select, button {  
    padding: 12px;  
    font-size: 16px;  
  }  
}
```

2. Commit and push:

Bash



```
git add css/responsive.css  
git commit -m "feat: optimized mobile input touch targets and  
padding"  
git push origin main
```

Developer B (Friend)

- **What to do:** Implement CSS `transform: scale()` scaling on the ID preview card container for mobile views.
- **Where to do it:** `css/id-card.css` and `css/responsive.css`.
- **How to do it (Step-by-Step):**

1. Add responsive scale rules for the preview container:

CSS



```
@media (max-width: 480px) {  
  #id-card-preview {  
    transform: scale(0.8);  
    transform-origin: top center;  
  }  
}
```

2. Commit and push:

Bash



```
git add css/id-card.css css/responsive.css  
git commit -m "feat: added css transform scaling for mobile id  
card previews"  
git push origin main
```

- **Weekend Sync:** Test responsiveness live using Chrome DevTools Device Toolbar or connected mobile devices.

Week 8: Code Refactoring, Cleanup & GitHub Pages Deployment (v2.0)

- **Aim of the Week:** Organize assets, clean up redundant code, and deploy the website live to the public using GitHub Pages.
- **Potential Roadblock:** Broken relative asset paths on production servers.
- **Resolution / Command:** Ensure all stylesheet and script references use relative paths (e.g., `css/style.css` instead of absolute system paths).

Developer A & B (Joint Execution)

- **What to do:** Audit folder structures, clean up commented-out legacy code, and configure GitHub Pages publishing.
- **Where to do it:** Repository Settings & Codebase root.
- **How to do it (Step-by-Step):**

1. Review all files and remove unused code blocks or debugging `console.log()` statements.

2. Stage, commit, and push final code:

Bash



```
git add .  
git commit -m "refactor: cleaned up legacy code and verified  
relative asset paths"  
git push origin main
```

3. Go to your GitHub repository **Settings** > **Pages**. Under *Build and deployment*, select **Deploy from a branch**, choose the `main` branch, and select `/ (root)` folder. Click **Save**.

4. Wait 2 minutes, then visit your live production URL: `[https://YOUR-USERNAME.github.io/school-website/](https://YOUR-USERNAME.github.io/school-website/)`

- **Weekend Sync:** Celebrate! Test the live public URL together and review your progress from week 1.

MONTHS 3 TO 6: Overview of Advanced Milestones

- **Month 3 (Weeks 9–12) — Backend Server & Authentication:**

- *Developer B* initializes a Node.js Express server (`server.js`), configures secure password hashing (`bcryptjs`), and connects MongoDB Atlas cloud database clusters using Mongoose schemas.
- *Developer A* builds admin login views (`login.html`) and connects frontend forms to backend REST API endpoints.

- **Month 4 (Weeks 13–16) — Batch Spreadsheet Uploads & QR Code Security:**

- *Developer B* integrates the SheetJS (`xlsx`) library to parse uploaded CSV/Excel class lists in-browser and embeds unique scannable QR verification hashes onto ID cards.
- *Developer A* designs back-side card print layouts and batch pagination tables.

- **Month 5 (Weeks 17–20) — Security Hardening, Compression & Beta Testing:**

- *Developer B* implements Helmet.js security headers, express rate-limiting, and client-side image compression scripts.
- *Developer A* manages external beta testing distribution and logs bugs inside GitHub Issues.

- **Month 6 (Weeks 21–24) — Polish, Dark Mode & Portfolio Release (v3.0):**

- *Developer B* implements a global Dark Mode toggle using CSS variables and local storage persistence.
- *Developer A* finalizes README documentation and tags the final release (`v3.0-stable`).

What specific file or code block from **Month 1, Week 1** would you like to verify or test first in your repository today?